Highlights

Diagnose locally, escalate rarely: bandwidth-efficient plant disease recognition with foundation models on embedded edge–cloud hardware

Youssef Natij, Hajar El Karch, Abdelkader Mezouari

- An edge model escalates only uncertain field images to a foundation-model ensemble
- 18.6 % escalation retains 94.6 % of cloud accuracy with 5.4× less uplink
- Edge-model capacity, not router complexity, sets the trade-off
- A 0.9-billion-parameter ensemble runs on an end-of-life 4 GB Jetson Nano
- A seven-day, four-farm replay on the devices measures up to 8× less uplink

Diagnose locally, escalate rarely: bandwidth-efficient plant disease recognition with foundation models on embedded edge–cloud hardware

Youssef Natij[a,*], Hajar El Karch[b], Abdelkader Mezouari[a]

[a] *Scientific Research and Innovation Laboratory, Higher School of Technology, Ibn Tofail University, Kenitra, Morocco*
[b] *Laboratory of Advanced Systems Engineering, National School of Applied Engineering, Ibn Tofail University, Kenitra, Morocco*

Abstract

Image-based plant disease diagnosis in the field faces a dilemma: lightweight models run on low-cost farm devices but misdiagnose about one field photograph in three, whereas foundation-model ensembles approach expert accuracy but need every photograph uploaded over weak, metered rural links. We study selective edge–cloud inference as the middle ground, evaluated by uplink traffic, the binding constraint on farms. On PlantWild (18 542 in-the-wild images, 89 classes), a MobileNetV3-Large edge classifier escalates an image to an ensemble of three frozen foundation backbones only when the gap between its two most probable classes falls below a threshold calibrated on validation data. The rule escalates 18.6 % of test images and retains 94.6 % of always-cloud accuracy (75.82 % vs. 80.17 % top-1), cutting uplink traffic 5.4-fold. Edge-model capacity, not router sophistication, sets this operating point: a weaker edge model needs 43.5 % escalation for the same target, and a cloud-supervised verifier ranks useful escalations worse than the margin (AUC 0.731 vs. 0.810) and adds an always-on message. We then deploy the system on a Raspberry Pi 5 and an end-of-life 4 GB Jetson Nano, porting the ensemble through framework-free, chunk-by-chunk TensorRT inference. In a replayed seven-day, four-farm deployment, it uploads 5.4× less than always-cloud, answers on the farm in 77 ms (median) and confirms each escalated image with 6.9 s of Nano compute; distilling the ensemble into the

*Corresponding author.
Email address: youssef.natij@uit.ac.ma (Youssef Natij)

edge model raises the measured saving to 8.0×. Selective escalation thus makes foundation-model accuracy affordable on rural links with commodity embedded hardware.

Keywords: plant disease recognition, edge computing, selective inference, foundation models, bandwidth efficiency, embedded systems

1. Introduction

Pathogens and pests destroy a substantial fraction of the global harvest of major food crops every year (Savary et al., 2019), and early, accurate diagnosis is what allows a grower to respond before an outbreak spreads. Smartphone and camera-based diagnosis has therefore become one of the most studied applications of deep learning in agriculture (Kamilaris and Prenafeta-Boldú, 2018; Mohanty et al., 2016; Ferentinos, 2018). Yet the field has long been flattered by laboratory benchmarks: models that exceed 99 % accuracy on curated leaf images such as PlantVillage (Hughes and Salathé, 2015) lose much of that accuracy on photographs taken in real fields, where backgrounds are cluttered, lighting is uncontrolled and symptoms are small (Barbedo, 2018; Noyan, 2022; Ahmad et al., 2023; Wu et al., 2023). The PlantWild benchmark (Wei et al., 2024b) made this gap measurable at scale, with 18 542 in-the-wild images spanning 89 crop–disease classes.

Two families of models now address this setting, and they sit at opposite ends of a deployment trade-off. Frozen *foundation-model* features — self-supervised DINOv2 and DINOv3 and language-supervised CLIP (Oquab et al., 2024; Siméoni et al., 2026; Radford et al., 2021) — are remarkably robust to field conditions; in prior work we showed that fusing three such backbones reaches 80.17 % top-1 on PlantWild without any text supervision (El Karch et al., 2025). Such ensembles, however, total close to a billion parameters and must run on a server. Lightweight convolutional networks such as MobileNetV3 (Howard et al., 2019) run comfortably on a farm-grade device but fall more than ten points short in the wild. A grower must thus choose between an accurate model that requires connectivity and a local model that is frequently wrong.

Connectivity is precisely what rural agriculture lacks. Cellular uplinks on farms are slow, intermittent and metered (Wolfert et al., 2017; O'Grady et al., 2019), and every photograph sent to a server costs airtime, energy and money. Uploading every image is the most expensive way to use a scarce resource,

yet it is the implicit assumption of most cloud-based diagnosis systems. Edge computing (Shi et al., 2016) suggests the alternative: process data where it is produced and send only what cannot be handled locally.

This paper studies *selective escalation*: an edge model diagnoses every image locally and forwards an image to the cloud ensemble only when it is uncertain. The idea is old in machine learning — classifier cascades (Viola and Jones, 2001), the reject option (Chow, 1970) and early exiting (Teerapittayanon et al., 2016) all exploit it — but its evaluation has centred on computation and latency. For connected agriculture the binding constraint is the uplink, so we make *bytes uploaded per diagnosis* the primary cost axis and ask three questions:

1. How much uplink traffic can selective escalation save while retaining most of the cloud ensemble's accuracy on in-the-wild images?
2. What determines the achievable operating point: the edge model, or the sophistication of the escalation rule?
3. Does the approach survive contact with real embedded hardware, including a cloud node small enough to sit on a farm or in a cooperative?

Contributions.

- **A bandwidth-centred evaluation of selective edge–cloud inference for plant disease recognition.** A MobileNetV3-Large edge model with a one-line margin rule, its threshold calibrated on validation data, retains 94.6 % of the ensemble's accuracy on held-out test images while escalating 18.6 % of them, a 5.4-fold reduction in uplink traffic (Section 6.2).

- **Evidence that edge capacity, not router cleverness, sets the operating point.** Upgrading the edge model halves the required escalation rate (43.5 % → 20.5 %), and distilling the ensemble into the same edge model raises the uplink saving measured on the devices from 5.4- to 8.0-fold, whereas a learned edge-side router saves only two points of escalation and a cloud-supervised verifier does worse than the scalar margin while adding a permanent message overhead (Sections 6.1–6.4 and 6.7).

- **A recipe for running foundation models on end-of-life embedded hardware.** The edge runs on a Raspberry Pi 5, and the 0.9-billion-parameter ensemble runs on a 4 GB Jetson Nano whose last software

release predates every framework the ensemble needs. Five principles make this possible — decoupling the model from its framework, compiling within the memory ceiling, choosing precision by measurement, chunk-major batched inference and stage-by-stage verification — and the board serves an escalated image with 49 J of energy, with board memory peaking at 2.32 GB (Section 5 and Appendix A).

- **A trace-driven, hardware-in-the-loop emulation of a multi-farm week.** Four virtual farms with distinct crop portfolios replay a seven-day capture schedule against the real devices, yielding measured per-image latency, per-farm upload volume, cloud load and device energy, for the original and an improved configuration (Sections 6.6 and 6.7).

2. Related work

2.1. Plant disease recognition in the wild

Early deep-learning studies reported near-perfect accuracy on laboratory images (Mohanty et al., 2016; Ferentinos, 2018), but subsequent work traced much of that performance to dataset bias and showed sharp drops under field conditions (Barbedo, 2018; Noyan, 2022; Ahmad et al., 2023). Field-oriented datasets such as PlantDoc (Singh et al., 2020) and, at a larger scale, PlantWild (Wei et al., 2024b) now drive the area, together with multimodal baselines that inject textual disease descriptions (Wei et al., 2024a). Recent work combines foundation backbones (Jiang et al., 2025; Salman et al., 2025) or compresses ensembles for deployment (Hasan et al., 2025). Our cloud model is the three-backbone QualityGate fusion of El Karch et al. (2025); this paper does not propose a new recogniser but asks how to deliver such a recogniser's accuracy to farms that cannot afford to upload every image.

2.2. Edge–cloud collaborative inference

Collaborative inference distributes a model's computation between a device and a server. Neurosurgeon partitions a single network at the layer that minimises latency or energy (Kang et al., 2017); distributed DNNs and early-exit networks attach classifiers to intermediate layers so that easy inputs leave early (Teerapittayanon et al., 2016, 2017); split computing compresses the intermediate tensor sent over the network (Matsubara et al., 2022). These

methods co-design a single model. We study the complementary *model-cascade* setting, in which the edge and cloud models are independent and the cloud model is a frozen ensemble that cannot be partitioned or retrained for the edge — the practical situation when the best available recogniser is a set of foundation backbones.

2.3. Cascades, selective prediction and escalation rules

Deciding which inputs deserve the expensive model is a selective prediction problem (Chow, 1970; Geifman and El-Yaniv, 2017). The maximum softmax probability is a strong baseline for detecting likely errors (Hendrycks and Gimpel, 2017), although modern networks are miscalibrated (Guo et al., 2017). Learned deferral policies have been proposed for cascades (Wang et al., 2018) and for edge devices that query a cloud model (Kag et al., 2023). We compare scalar uncertainty signals against both edge-side learned routers and a cloud-supervised verifier, and we account for the bandwidth that a learned verifier itself consumes — a cost that latency-centred evaluations omit.

2.4. Edge AI in agriculture

Surveys of deep learning in agriculture (Kamilaris and Prenafeta-Boldú, 2018) and of smart farming (Wolfert et al., 2017) identify connectivity and on-farm computation as recurring obstacles, and edge computing has been argued to be a natural fit for agricultural data (O'Grady et al., 2019). The disease-recognition studies discussed in Section 2.1, however, evaluate either a server-side or a fully on-device classifier and report accuracy and, at most, computational cost; none of them measures the uplink traffic that a deployment would generate. Our deployment also differs in placing the large model on an embedded node rather than in a data centre.

3. Materials and methods

3.1. Dataset and protocol

We use PlantWild (Wei et al., 2024b): 18 542 field photographs of 89 classes spanning healthy and diseased leaves of more than thirty plant species. We follow the official split procedure of Wei et al. (2024a) (fixed seed, 80 % train-validation and 20 % test, with 12.5 % of train-validation held out for validation), giving 13 045 training, 1 820 validation and 3 677 test images. All accuracies are top-1 on the 3 677-image test split. Training, offline evaluation and the ONNX export ran on a workstation with an NVIDIA GeForce RTX 3070 (8 GB), an AMD Ryzen 7 7700 CPU and 64 GB of RAM.

3.2. Cloud model: frozen foundation-backbone fusion

The cloud model is the QualityGate fusion of El Karch et al. (2025) (Fig. 1, right). Three frozen backbones each produce a global image descriptor: DINOv2 ViT-L/14 at 336 px (Oquab et al., 2024), DINOv3 ViT-L/16 at 336 px (Siméoni et al., 2026), and the CLIP ViT-L/14 image encoder at 224 px (Radford et al., 2021). Each descriptor is ℓ_2-normalised, the three are concatenated into a 2 816-dimensional vector and the result is normalised again, giving $\hat{x}$. The QualityGate head mixes a linear classifier with a prototype classifier (K=8 prototypes $p_{c,k}$ per class) through a learned per-class gate:

$$z_c = g_c \left(w_c^\top \hat{x} + b_c \right) + (1 - g_c) \, \frac{1}{T} \max_k \cos(\hat{x}, p_{c,k}) \,, \qquad g_c = \sigma(\gamma_c), \tag{1}$$

with a learned temperature T. Only the head is trained (2.3 M parameters, 50 epochs); the backbones total about 0.9 B frozen parameters. The model reaches 80.17 % top-1 and 77.49 % macro-F1 on the test split, and we refer to it as *always-cloud*.

3.3. Edge models

We consider three ImageNet-pretrained networks that fit a farm-grade CPU: MobileNetV3-Small, MobileNetV3-Large (Howard et al., 2019) and EfficientNet-B0 (Tan and Le, 2019). Each is fine-tuned end to end at 224×224 for 30 epochs with AdamW (Loshchilov and Hutter, 2019) (learning rate $5{\times}10^{-4}$, weight decay 10^{-4}, batch 64, cosine schedule with 5 % linear warm-up), focal loss (Lin et al., 2017) with γ=2 and label smoothing 0.1, and random resized crops (scale 0.6–1.0), horizontal flips and colour jitter. The checkpoint with the best macro-F1 on a stratified 10 % hold-out of the training split is kept, so that the 1 820 validation images remain unseen until the escalation threshold is calibrated on them.

3.4. Margin-based escalation

Let $p_{(1)} \geq p_{(2)}$ be the two largest softmax probabilities of the edge model for image x. The edge escalates x to the cloud if and only if its *margin* is small,

$$m(x) = p_{(1)}(x) - p_{(2)}(x) < \tau, \tag{2}$$

and otherwise returns its own prediction. The rule has a single parameter, costs one subtraction, and requires no training. Sweeping τ traces

an accuracy–escalation curve; we also evaluate the maximum probability $p_{(1)}$ and the predictive entropy as alternative signals, two learned edge-side routers, and an *oracle* that escalates exactly those images the edge gets wrong and the cloud gets right — an upper bound for any escalation rule.

Threshold calibration. The deployed threshold is fixed before any test image is seen: τ is the smallest value whose *validation* accuracy reaches 95 % of the cloud model's validation accuracy. For MobileNetV3-Large this gives $\tau = 0.138$, which escalates 19.3 % of validation images. The same τ is then applied unchanged to the test split and to the deployment. Full accuracy–escalation curves, which do not depend on any particular τ, are used only to compare routers and edge models.

3.5. Bandwidth model

Let r be the escalation rate and S the mean size of an escalated image. The mean uplink traffic per diagnosis is

$$\bar{B}_{\text{margin}} = r\,S, \qquad \bar{B}_{\text{verifier}} = M + r\,S, \qquad \bar{B}_{\text{cloud}} = S, \tag{3}$$

where M is the size of any message a router sends for *every* image (Section 3.6). The reduction factor of margin escalation over always-cloud is therefore $\bar{B}_{\text{cloud}}/\bar{B}_{\text{margin}} = 1/r$, independent of S. To express traffic in absolute terms we set S to the measured mean size of the escalation payload used in the deployment (Section 4.3): a 448 px JPEG at quality 90, $S = 60.2$ KB over the 3 677 test images. Offline predictions and on-device measurements (Section 6.6) are therefore in the same units. Sizes use binary prefixes throughout (1 KB = 1024 bytes, 1 MB = 1024 KB).

3.6. Cloud-supervised verifier

To test whether cloud-side information can improve escalation, we train a verifier: a multilayer perceptron that runs on the cloud and receives, for every image, the edge model's 1 280-dimensional penultimate embedding, its 89-dimensional softmax and its uncertainty scores. It is trained to predict one of three targets — *useful escalation* (edge wrong, cloud right), *cloud disagrees* with the edge, or *cloud correct* — and the image is escalated when the verifier's score exceeds a threshold. Because the verifier needs the embedding of every image, it adds an always-on message of $M = 2 \times (1\,280 + 89)$ bytes $= 2.67$ KB in half precision.

3.7. Success criterion

Before running experiments we fixed a target: an operating point is acceptable (*GO*) if it reaches at least 95 % of always-cloud accuracy (76.17 % top-1) while escalating at most 30 % of images.

4. Embedded system implementation

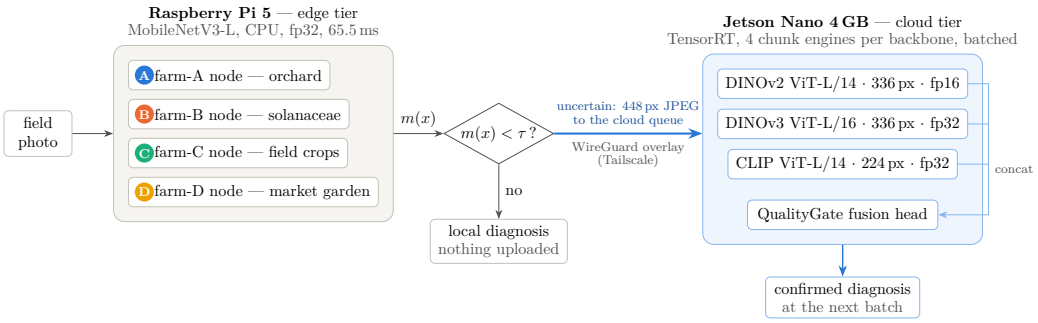


Figure 1: System overview. Each farm node runs the edge model on every image, answers the farmer immediately, and computes the margin $m(x)$ between its two most probable classes. Only uncertain images ($m < \tau$) are uploaded, resized to a 448 px shorter side, to the cloud node's queue; the cloud processes its queue in batches, backbone by backbone with one chunk engine in memory at a time, and returns the ensemble's diagnosis (Section 5.4). All traffic travels over an encrypted WireGuard overlay. In our testbed the four farm nodes are independent processes on one Raspberry Pi 5 (Section 4.4).

4.1. System architecture

Figure 2 shows the system as deployed. It has three tiers, each a set of independent services that communicate only through small HTTP/JSON interfaces. The *edge tier* runs one service per farm (`plant-edge-farm@A...D`, managed by systemd on the Raspberry Pi 5); the *cloud tier* runs a single service (`plant-cloud-trt` on the Jetson Nano); and an *operations tier* on a workstation drives the experiment, records every event and displays the fleet's state. Four paths connect them. A capture is sent to its farm's edge service (path 1), which answers immediately with its own diagnosis, margin and upload size (path 2). Only if the margin is below τ does the edge service also resize the image and place it on the cloud's queue (path 3), so the answer path never waits for the cloud. Twice a day the queue is processed as one batch and the ensemble's confirmed diagnoses are returned (path 4). Because

the tiers share nothing but these interfaces, a farm can be added by starting one more edge service, and the cloud tier can move from the Nano to any larger machine without changing the edge.

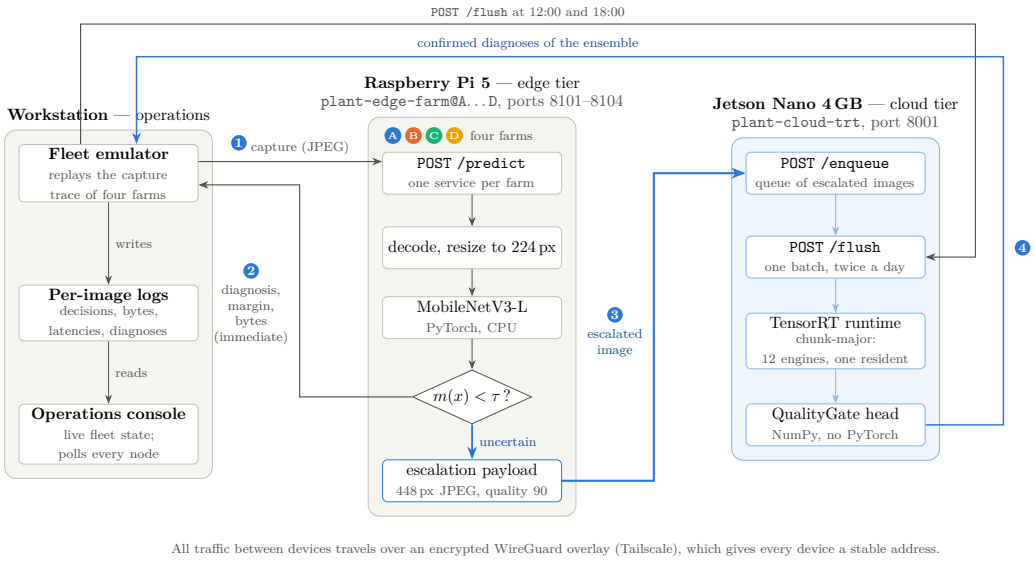


Figure 2: System architecture as deployed. Three tiers of independent services communicate through HTTP/JSON interfaces: (1) a capture is sent to its farm's edge service, (2) which answers immediately; (3) uncertain images are additionally queued on the cloud node, (4) which processes its queue twice a day and returns the ensemble's confirmed diagnoses. In our testbed the four edge services share one Raspberry Pi 5 (Section 4.4) and the operations tier runs on the workstation.

4.2. Hardware and software

Table 1 summarises the testbed. The edge is a Raspberry Pi 5 running the edge model in PyTorch on its CPU. The cloud node is a 4 GB Jetson Nano, a low-cost embedded GPU module, chosen deliberately: if a three-backbone foundation ensemble can be served from a device of this class, the "cloud" in our architecture can be a cooperative- or farm-level box rather than a data centre. The nodes communicate over a WireGuard-based overlay network (Donenfeld, 2017) (Tailscale), which encrypts all traffic and gives each device a stable address regardless of its physical network. Both servers expose a small HTTP/JSON interface in which images travel as base64-encoded JPEG. All byte counts in this paper are those of the JPEG payload; the base64 text

encoding of our prototype adds a third on top of them, an overhead that a binary upload removes and that leaves every reduction factor unchanged.

Table 1: Embedded testbed.

	Edge	**Cloud**
Device	Raspberry Pi 5 Model B	Jetson Nano Developer Kit
Processor	4× Cortex-A76, 2.4 GHz	4× Cortex-A57 + 128-core Maxwell GPU
Memory	16 GB	4 GB (shared CPU/GPU)
OS	Debian 13	Ubuntu 18.04 (JetPack 4.6.1, L4T R32.7.1)
Runtime	Python 3.13, PyTorch 2.6 (CPU)	Python 3.6, TensorRT 8.2.1, CUDA 10.2
Model	MobileNetV3-L, fp32	DINOv2-L (fp16), DINOv3-L and CLIP-L (fp32); 4 chunk engines each
Power mode	default (`ondemand`)	MAXN

4.3. Edge node

Each edge node is a stateless HTTP service. For every image it decodes the JPEG, applies the edge model, computes the margin and, if $m(x) < \tau$, forwards the image to the cloud node; the uploaded byte count is recorded per image once the cloud acknowledges it. Escalation is *deferred*: the node places the image on the cloud node's queue and immediately returns its own label, so the farmer is never blocked by the cloud; the ensemble's diagnosis follows when the queue is processed (Section 5.4).

Escalation payload. Field photographs are often far larger than the cloud model needs: some PlantWild images exceed 12 megapixels, while the cloud backbones operate at 224–336 px. Uploading at the camera's resolution therefore wastes most of the escalated bytes. The edge instead resizes an escalated image so that its shorter side is 448 px and encodes it as JPEG at quality 90. Table 2 shows the trade-off measured with the reference cloud pipeline: at 448 px the cloud returns the same diagnosis as from the original photograph for 97.7 % of images, for 2.5× fewer bytes than a full-resolution upload. The always-cloud baseline in the deployment uses the identical payload, so the measured reduction factor isolates the effect of selective escalation.

Table 2: Escalation payload policy (300 test images, reference cloud pipeline). Agreement is the fraction of images for which the cloud's diagnosis from the payload equals its diagnosis from the original file (162 KB on average).

Payload	**KB / escalation**	**Agreement**	**Cloud top-1**
Full resolution, JPEG q90	149.0	—	81.0 %
Shorter side 448 px, q90	**59.7**	**97.7 %**	**82.0 %**
Shorter side 336 px, q95	58.2	96.3 %	80.3 %
Shorter side 336 px, q90	42.7	95.7 %	80.0 %
Shorter side 224 px, q90	23.1	93.7 %	79.3 %

If the cloud is unreachable, the node falls back to its own prediction and records the failure, so that an outage degrades accuracy rather than availability. The deployment uses the validation-calibrated $\tau = 0.138$ (Section 3.4).

4.4. Trace-driven emulation of a multi-farm week

A field deployment spans days, with hours between captures. To study that regime within an experimental session we use trace-driven, hardware-in-the-loop emulation rather than simulation: a recorded workload (the test images, arranged into a week of captures) is replayed through the real devices and the real network, so that every inference, every transfer and every byte is executed and measured rather than modelled. Only the clock is compressed — idle time between captures is skipped — and we refer to its time axis as simulated hours and days. We call this experiment the *replay* below.

Virtual farms. Four farm nodes run as independent edge processes on the Raspberry Pi 5, each with its own identifier, port, routing decisions and connection to the cloud node; the cloud therefore serves four distinct clients. Farms differ in what they grow: the 89 classes are grouped by crop into an orchard (apple, banana, blueberry, cherry, citrus, coffee, grape, maple, peach, plum, raspberry, strawberry; 1 360 test images), a solanaceous-crop farm (bell pepper, eggplant, potato, tobacco, tomato; 701), a field-crop farm (bean, corn, garlic, ginger, rice, soybean; 707) and a market garden (basil, broccoli, cabbage, carrot, cauliflower, celery, cucumber, lettuce, squash, zucchini; 909). Together they cover the full 3 677-image test split exactly once.

Schedule. Each farm's images are spread evenly over the daylight hours (07:00–19:00) of seven simulated days, giving 84 capture hours per farm;

within an hour, requests from different farms are interleaved. One simulated hour lasts at least one second of wall-clock time and is stretched when its captures take longer, so the replay never drops or overlaps work.

What is and is not emulated. Requests are issued one at a time, so every edge latency is that of a single request on an otherwise idle device — exactly what a farm owning its own Pi would observe. Upload sizes, escalation decisions, diagnoses and per-stage latencies (edge inference, network, cloud inference) are measured for every image. What the replay compresses is idle time; what it emulates is the multiplicity of edge devices. Table 3 lists the provenance of every quantity reported in this paper.

Table 3: Provenance of the reported quantities.

Quantity	**Provenance**
Accuracy, escalation rate (offline)	measured on the PlantWild test split
Bandwidth reduction factor $1/r$	measured (independent of image size)
Absolute KB per image (offline)	escalation rate $\times$ measured mean payload S
Absolute KB per image (deployment)	measured JPEG bytes of every upload
Edge latency	measured on the Raspberry Pi 5
Cloud latency	measured on the Jetson Nano
Cloud memory and energy	measured on the Jetson Nano (`tegrastats`: RAM, board input power)
Edge energy	measured on the Raspberry Pi 5 (power-management IC: SoC and memory rails, a lower bound)
Distillation and loop pilot	offline, on the workstation; test split as in Section 3.1
Network latency	measured over the overlay; local network, i.e. a best case for rural links
Four farms	four processes on one physical Pi (emulated multiplicity)
Seven days	simulated clock; idle time compressed, work not compressed

5. Running a foundation-model ensemble on end-of-life embedded hardware

The Jetson Nano is typical of hardware that farms, cooperatives and agricultural schools already own or can buy cheaply, but that the software ecosystem has left behind. NVIDIA's last software release for it (JetPack 4.6) ships Ubuntu 18.04, Python 3.6, CUDA 10.2 and TensorRT 8.2, whereas the reference cloud implementation relies on PyTorch 2, `timm` and `open_clip`, none of which runs on that stack; DINOv3 in particular requires a far newer toolchain. Memory is the second wall: the board's 4 GB are shared by CPU and GPU, while the fp32 weights of the three backbones alone occupy about 3.4 GB. Rather than rebuild a framework stack on the device, we treated the port as an engineering problem and solved it with five principles, each of which transfers to other legacy accelerators. Table 4 summarises the decisive obstacles; Appendix A lists all of the adaptations the port required.

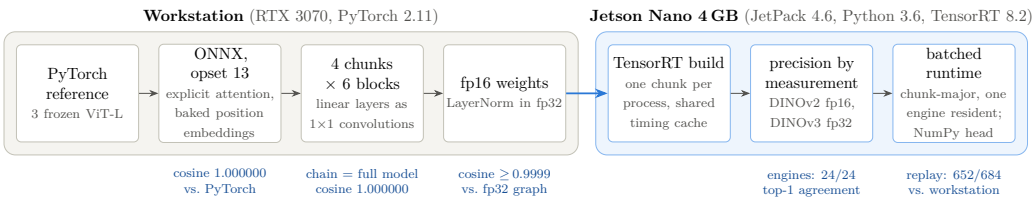


Figure 3: Porting the cloud ensemble to the Jetson Nano. The model is decoupled from its framework on the workstation and compiled chunk by chunk on the device; the blue labels give the fidelity check passed at each stage (Table 5).

5.1. Decouple the model from its framework

On the workstation, each backbone is exported to ONNX (Bai et al., 2019) at opset 13, the newest opset that TensorRT 8.2 parses (Fig. 3). Fused scaled-dot-product attention has no opset-13 equivalent, so attention is traced in its explicit $\mathrm{softmax}(QK^\top/\sqrt{d})V$ form, which is mathematically identical. Operations that depend on the input shape — DINOv2's bicubic resizing of its position grid, DINOv3's rotary position embedding — are evaluated once for the fixed input size and stored as constants, and an export is rejected if any operator unsupported by TensorRT 8.2 remains. Image preprocessing (bicubic resize, centre crop, normalisation) and the QualityGate head of Eq. (1) are re-implemented in NumPy, so the serving process on the Nano needs neither PyTorch nor its dependencies.

Table 4: The decisive obstacles of the port, their causes and the adaptations that resolved them (all items: Appendix A).

Obstacle	**Cause**	**Adaptation**	**Evidence**
Reference server cannot run	JetPack 4: Python 3.6, CUDA 10.2	ONNX export on the workstation; torch-free NumPy server	graphs cosine 1.000000; preprocessing bit-identical
Operators rejected by TensorRT 8.2	fused attention, cubic resize, shape-dependent position embeddings	explicit attention; position embeddings baked as constants; constant folding	no unsupported operator left
A whole ViT-L cannot be compiled	ONNX parser alone peaks at 2.75 GB	fp16 weights; four chunks of six blocks	chained chunks cosine 1.000000
969 MB of device memory per chunk	weights fused into one opaque kernel graph	every linear layer as a 1×1 convolution	engine $612 \rightarrow 157$ MB
NaN features from DINOv3	residual stream 1.6×10^5, above the fp16 range	precision chosen per backbone from measured ranges	DINOv3 engine cosine 1.0000
Only 9 of 12 chunks fit	$\approx$0.35 GB per loaded chunk	chunk-major batched inference, deferred escalation	peak 2.32 GB of 3.86 GB
Residual on-device deviation	Pillow 8.4 decodes JPEGs differently	isolated and measured, not correctable on JetPack 4	652/684 replay agreement

5.2. Compile within the memory ceiling

A whole ViT-L cannot be compiled on the Nano: TensorRT's ONNX parser alone peaks at 2.75 GB of RAM for a 0.6 GB fp16 graph, before the builder has started (Fig. 4b). Each backbone is therefore exported as four consecutive chunks of six transformer blocks (the first also holding the patch embedding, the last the pooling and projection), converted to fp16 weights with LayerNorm arithmetic kept in fp32, and compiled into a TensorRT engine (NVIDIA Corporation, 2022) one chunk at a time; at run time the chunks are chained and compute exactly the full forward pass (cosine 1.000000 against the unsplit model). A second, less visible limit followed: TensorRT 8.2 fuses transformer layers whose weights enter through matrix products into one opaque kernel graph, which held 969 MB of device memory for six blocks — about 13 bytes per parameter. Expressing every linear layer as the equivalent

1×1 convolution keeps the weights in TensorRT's native layers and shrinks the engine for the same six blocks from 612 to 157 MB.

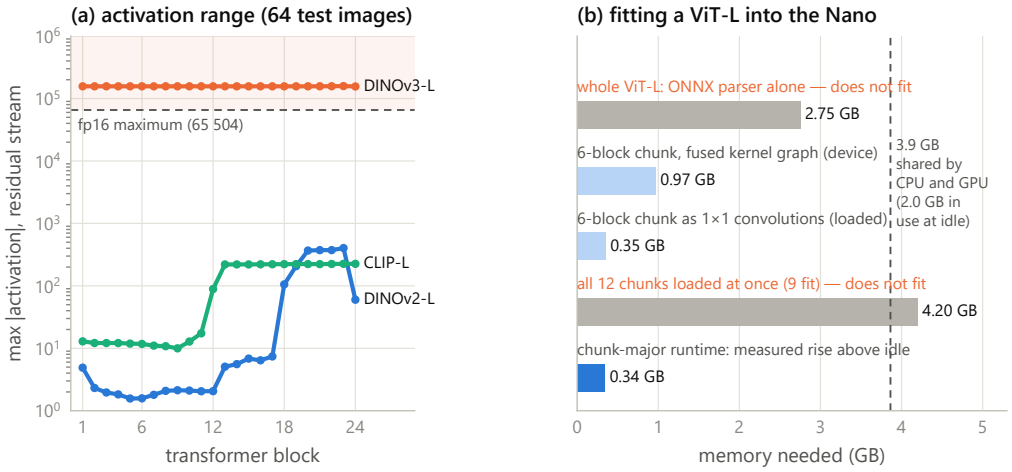


Figure 4: Why the port takes the shape it does. (a) Largest residual-stream activation per transformer block over 64 test images (fp32 reference): DINOv3 carries activations of about 1.6×10^5 from its first block onwards, above the largest fp16 value, whereas DINOv2 and CLIP stay below 500. (b) Memory needed at each step of the port against the board's shared CPU/GPU memory; grey bars did not fit.

5.3. Choose precision by measurement

Half precision halves weights and load time, but it does not suit every backbone. Our first fp16 build of DINOv3 produced NaN features. The natural suspect, LayerNorm overflowing in fp16, was wrong: pinning LayerNorm, and then every non-convolution layer, to fp32 changed nothing. Profiling the reference model (Fig. 4a) located the cause: DINOv3's residual stream carries activations of up to 158 259 from its first block onwards, beyond the largest fp16 value (65 504), whereas DINOv2 and CLIP peak at 403 and 225. DINOv2 therefore runs in fp16 and DINOv3 in fp32. CLIP was moved to fp32 as well while we investigated a deviation that we later traced to the image library rather than to precision (Section 5.5). A later check confirmed that fp32 is unnecessary for CLIP: fp16 CLIP engines reproduce the fp32 ones to a cosine of at least 0.9998 on the 24 reference images, with the same pipeline agreement, while cutting CLIP's compute from 1 613 to 1 158 ms per image and its loading from 19 to 10 s. The replay reported here used fp32 CLIP; fp16 is the better choice for new deployments.

5.4. Run within the memory ceiling: chunk-major batched inference

Even in compact form, a loaded fp16 chunk engine occupies about 0.35 GB of the Nano's shared memory (157 MB of weights on the device plus the runtime's host-side copy and scratch), and we measured that at most nine of the twelve chunks can be resident at once, even with a single activation buffer shared by all of them. The full ensemble therefore cannot answer requests synchronously on this device. We resolve the conflict with a design that matches how diagnoses are used on farms: an escalated image is *queued* on the cloud node, and the edge returns its own label to the farmer immediately. The cloud node processes its queue in batches, chunk by chunk: it loads the first chunk of DINOv2, passes every queued image through it, frees it, loads the next chunk, and so on through DINOv3 and CLIP, keeping the intermediate activations (about 2 MB per image) in host memory, and finally applies the head. At no time is more than one chunk engine resident. In the replay the queue is processed twice a day, after the 12:00 and 18:00 captures, so every escalated image receives the ensemble's diagnosis on the day it was taken; the batch interval trades confirmation delay for model-loading overhead. The server also implements a synchronous mode with all engines resident, intended for cloud nodes with more memory; we did not test it.

To quantify the design on the device, we replayed 2 batches of 48 escalated images — the median batch of the replay, with the same payload — while sampling RAM and the board's input power rail with `tegrastats` about eight times per second (Fig. 5). Memory use never exceeds 2.32 GB of the 3.86 GB (1.97 GB at idle, including the operating system and the idle server): processing a batch adds only about 0.34 GB, the footprint of one chunk engine. The board draws 6.3 W on average while processing a batch (1.2 W at idle, 7.9 W at most), and a batch takes 6.1 min. Each escalated image costs 49 J of board energy including its share of model loading, or 40 J above the idle draw — about 14 Wh per thousand escalated images.

5.5. Verify every stage

A port is only useful if it computes the same function, so we verify each stage against the reference pipeline (Table 5). The exported graphs match PyTorch to a cosine similarity of 1.000000, the NumPy preprocessing reproduces the reference tensors exactly, and the NumPy head matches the original to within 3×10^{-6} in the logits. The reassembled pipeline agrees with the offline reference cloud predictions on 299 of 300 test images (99.7 %). On the Nano,

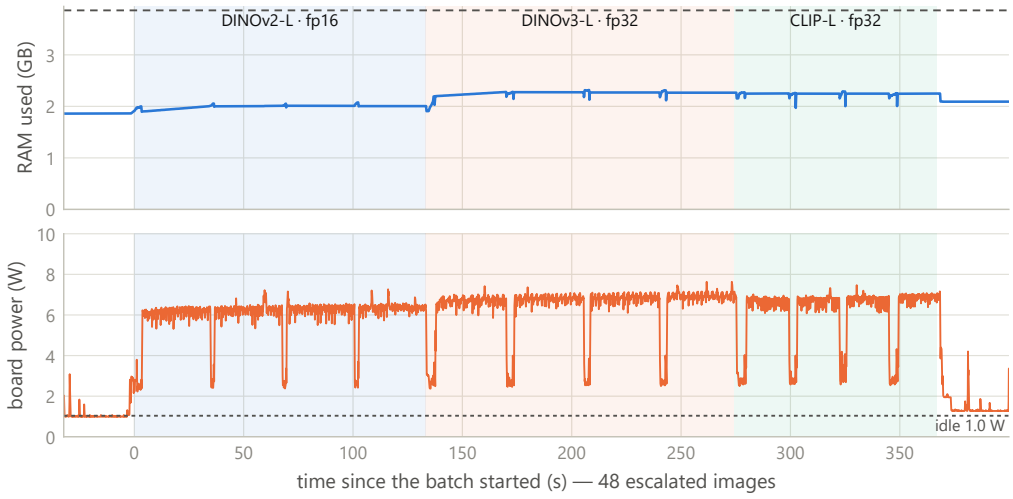


Figure 5: Memory and power of the Jetson Nano during one batch of 48 escalated images, measured with `tegrastats`. Shading marks the backbone being processed. Each backbone runs as four chunk engines loaded one at a time: the twelve dips in power are the twelve engine loads, between which the GPU processes all queued images through the resident chunk. The dashed line in the upper panel is the board's total shared memory; in the lower panel, the idle power draw.

fed the workstation's preprocessed tensors, the TensorRT engines reproduce the reference features to a cosine of 1.0000 (DINOv3, fp32), 1.0000 (CLIP, fp32) and 0.9965 (DINOv2, fp16) and the reference top-1 prediction on 24/24 images. The complete on-device pipeline agrees on 23/24: the residual deviation (minimum cosine 0.9474) does not come from TensorRT but from the image library — Pillow 8.4, the last release for JetPack 4's Python 3.6, decodes and resamples JPEGs slightly differently from the workstation's Pillow 12, by up to 0.59 normalised units on some pixels. Over the replay, the Nano's diagnosis matches the workstation's offline prediction for 652/684 escalated images. Isolating causes in this way — engines fed identical tensors, then the complete pipeline — is what separated a correctable precision problem (DINOv3) from an uncorrectable library difference (Pillow).

Table 5: Fidelity of the embedded cloud pipeline, verified stage by stage. Desktop checks compare against the PyTorch reference; Nano checks compare TensorRT outputs against reference features computed on the desktop.

Stage	**Check**	**Result**
ONNX export (3 backbones)	cosine vs. PyTorch, min over images	1.000000
Chunking + 1×1-convolution form	chained chunks vs. unsplit model, cosine (PyTorch and ONNX)	1.000000
fp16 weights, fp32 LayerNorm	cosine vs. fp32 graph, min	≥ 0.9999
NumPy preprocessing	max. abs. difference vs. torchvision	0
NumPy QualityGate head	max. abs. logit difference	2.9×10^{-6}
Full pipeline	agreement with offline cloud predictions	299/300 (99.7 %)
Edge node (Raspberry Pi 5)	label agreement with offline edge predictions	300/300
	margin, mean / max. abs. difference	$8{\times}10^{-4}$ / $7.8{\times}10^{-3}$
Nano engines, workstation-preprocessed inputs		
TensorRT fp16, DINOv2-L	cosine vs. reference, min / mean	0.9965 / 0.9979
TensorRT fp32, DINOv3-L	cosine vs. reference, min / mean	1.0000 / 1.0000
TensorRT fp32, CLIP-L	cosine vs. reference, min / mean	1.0000 / 1.0000
Engines + head	top-1 agreement with reference	24/24
Complete on-device pipeline (Nano image library)		
DINOv2-L / DINOv3-L / CLIP-L	cosine vs. reference, min	0.9955 / 0.9874 / 0.9474
Full pipeline (Nano)	top-1 agreement with reference	23/24
Full pipeline (Nano, replay)	agreement with offline cloud predictions, every escalated image	652/684

6. Results

6.1. Edge capacity determines the operating point

Table 6 and Fig. 6 compare the three edge models. With MobileNetV3-Small as the edge (61.08 % top-1), the margin rule reaches the 95 % target only at 43.5 % escalation, failing the pre-specified criterion. This failure is not a weakness of the margin rule: a learned router trained on the same edge outputs gained only two points (41.4 % escalation), and adding 1 024-dimensional DINOv2 image features to the router overfitted. For this edge model, every edge-side signal we tried saturated at an AUC of about 0.76–0.79 for predicting useful escalations — the information an edge-side router can see is bounded by the edge model itself.

Replacing the edge model changes the picture. MobileNetV3-Large raises edge accuracy by 8.2 points to 69.32 % and halves the required escalation to 20.5 %, comfortably passing the criterion. EfficientNet-B0 reaches the same edge accuracy (69.35 %) but needs slightly more escalation (21.7 %) and 1.8× the multiply–accumulate operations, so MobileNetV3-Large is our edge model for the remainder of the paper.

Table 6: Edge models on the PlantWild test split. "Escalation @95 %" is the smallest escalation rate at which margin escalation reaches 95 % of always-cloud accuracy (76.17 %).

Edge model	**Params**	**GMACs**	**Top-1**	**Esc. @95 %**	**Criterion**
MobileNetV3-Small	1.61 M	0.059	61.08 %	43.5 %	fail
MobileNetV3-Large	**4.32 M**	**0.225**	**69.32 %**	**20.5 %**	**pass**
EfficientNet-B0	4.12 M	0.400	69.35 %	21.7 %	pass

The oracle curve in Fig. 7 and the last column of Table 8 add a further observation. At 20 % escalation the oracle reaches 84.8 % with MobileNetV3-Large, *above* the always-cloud model: the edge and the cloud make partly different errors, so a perfect router would beat both. The gap between the practical rules and the oracle measures how much a better uncertainty signal could still gain; it does not depend on the cloud model, which is unchanged.

6.2. Accuracy–bandwidth trade-off

Table 7 and Fig. 7 place the operating points on the bandwidth axis. With the validation-calibrated threshold, margin escalation from MobileNetV3-Large reaches 75.82 % top-1 on the test split — 94.6 % of always-cloud accuracy — while escalating 18.6 % of images: 11.2 KB per diagnosis instead of

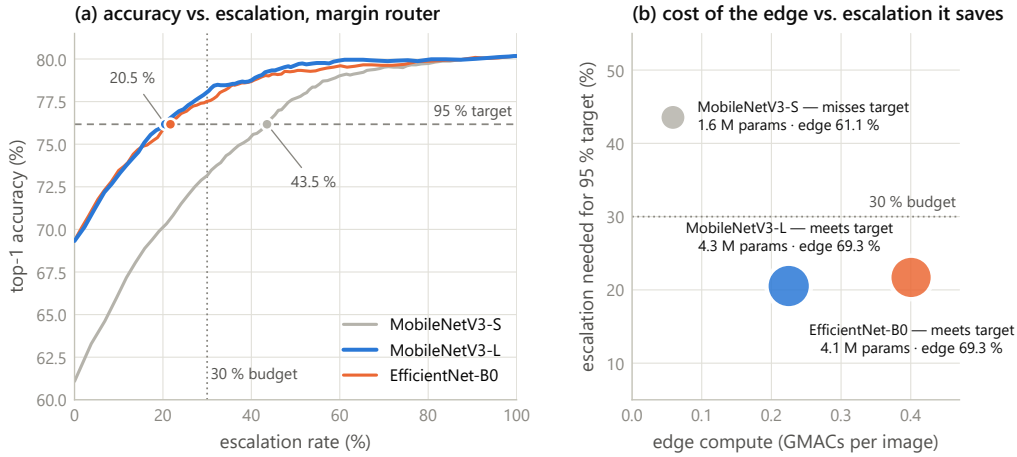


Figure 6: Edge capacity sets the operating point. (a) Accuracy versus escalation rate under margin escalation for the three edge models; dots mark where each first reaches the 95 % target. (b) Edge compute against the escalation each model needs to reach the target; bubble area is proportional to the parameter count. The dotted lines mark the 30 % escalation budget.

60.2 KB, a 5.38-fold reduction. On the validation split, where τ was set, the rule meets the criterion by construction (95.0 % of cloud accuracy at 19.3 % escalation); the 0.4-point shortfall on test is the honest generalisation gap of a threshold fixed in advance. Read directly off the test curve, the 95 % line is crossed at 20.5 % escalation (12.4 KB). The curve is markedly concave: the first escalated images are the most valuable. Escalating 30 % of images already closes 81 % of the gap between edge and cloud accuracy (Table 8), so the last fifth of that gap costs the remaining 70 % of the traffic. Table 8 shows that the three scalar signals are within 1.5 points of each other at every budget; we adopt the margin because it ties for best at the target and is the simplest to interpret.

Figure 8 follows every test image through the system. Of the 684 escalated images, 468 (68 %) were edge errors, so the margin spends uplink overwhelmingly where it is needed; the cloud rescues 289 of them and overturns 50 correct edge predictions, a net gain of 239 diagnoses. The figure also shows the limit of any edge-side signal: 660 edge errors are made with a confident margin and never leave the farm.

The rule also adapts across crops without any per-crop tuning (Fig. 9). Escalation ranges from 3 % of raspberry images to 36 % of bean images, and

Table 7: Operating points on the PlantWild test split (offline). Absolute bandwidth uses the measured mean payload S=60.2 KB; the reduction factor does not depend on S. The lower block reads both routers' test curves at the same accuracy, for comparison only.

Operating point	**KB/img**	**Escal.**	**Top-1**	**% cloud**
Edge only (MobileNetV3-L)	0.0	0 %	69.32 %	86.5 %
Margin escalation, τ=0.138 calibrated on validation	**11.2**	**18.6 %**	**75.82 %**	**94.6 %**
Always-cloud	60.2	100 %	80.17 %	100 %
Test curves read at 95 % of cloud accuracy				
Margin escalation	12.4	20.5 %	76.26 %	95.1 %
Cloud-supervised verifier	20.7	29.9 %	76.29 %	95.2 %

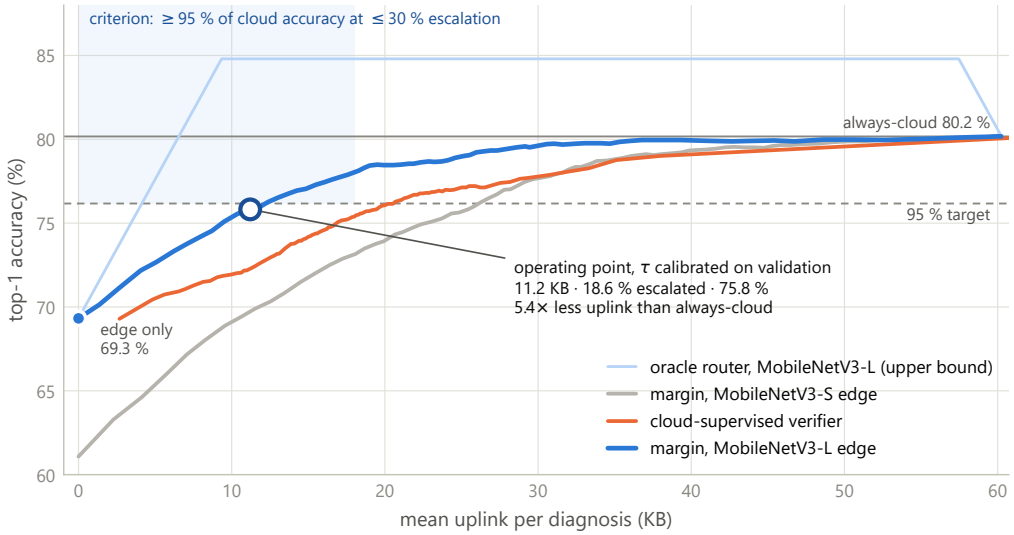


Figure 7: Accuracy versus mean uplink traffic per diagnosis (offline, test split, measured mean payload S=60.2 KB). The ring marks the operating point of MobileNetV3-L with the validation-calibrated threshold; the shaded region is the pre-specified success criterion; the light curve is the oracle router, an upper bound for any escalation rule.

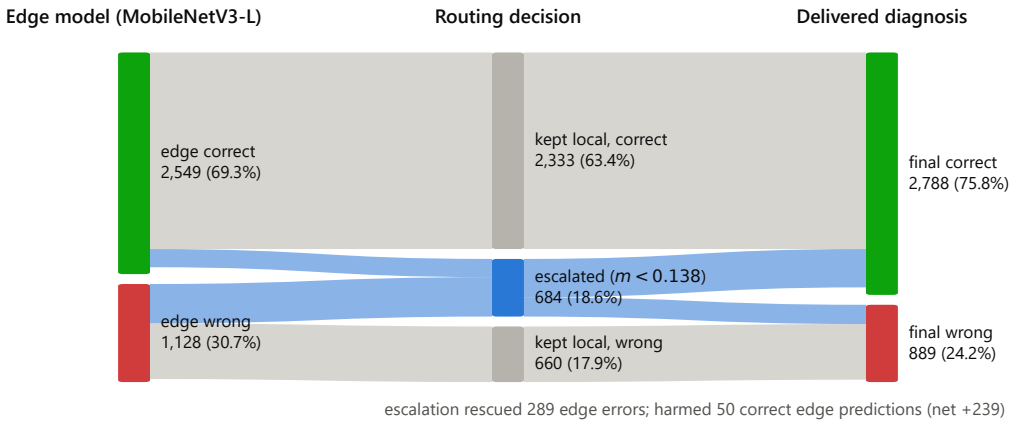


Figure 8: Where the 3 677 test images go at the calibrated threshold. Band widths are proportional to image counts; blue bands are escalated images. Escalation rescues 289 edge errors and overturns 50 correct edge predictions.

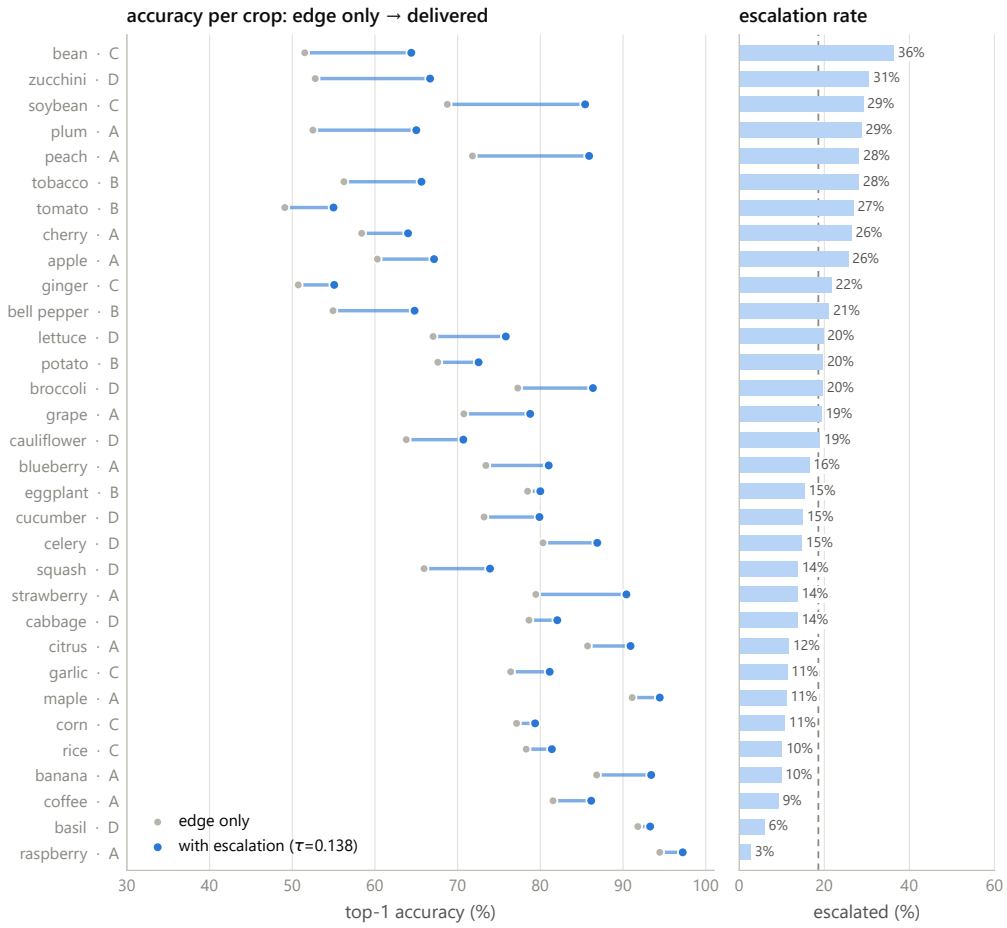
it is strongly anti-correlated with the edge model's accuracy on the crop (Pearson $r = -0.85$): bandwidth flows to the crops the edge finds hardest, which are also those that gain most — soybean (+16.7 points), peach (+14.1), zucchini (+13.9) and bean (+12.9).

Table 8: Top-1 accuracy of MobileNetV3-L escalation at fixed escalation budgets, by signal.

Escalation	**Max. probability**	**Margin**	**Entropy**	**Oracle**
5 %	71.99	71.85	71.85	73.92
10 %	72.91	73.21	72.69	79.90
15 %	74.41	74.08	73.95	84.42
20 %	76.07	75.96	75.82	84.80
25 %	77.32	77.05	76.77	84.80
30 %	77.97	78.13	77.51	84.80

6.3. A cloud-supervised verifier does not pay for itself

If edge-side signals are bounded by what the edge knows, a router that is trained with the cloud's answers might do better. It does not (Table 9, Fig. 10). The best verifier scores an AUC of 0.731 for identifying useful escalations, below the scalar $1 - m(x)$ at 0.810. On the bandwidth axis



Farm letters: A = orchard, B = solanaceae, C = field crops, D = market garden. Dashed line: overall escalation rate. Crops with ≥ 20 test images.

Figure 9: Per-crop accuracy before and after escalation (left) and the share of images escalated (right), at the calibrated threshold. Letters give each crop's farm in the replay (Section 4.4).

the verifier is dominated twice over: it needs more escalations for the same accuracy (29.9 % versus 20.5 %) and it adds its 2.67 KB message to every image. From Eq. (3), merely breaking even would require the verifier to escalate $M/S \approx 4.4$ points *fewer* images than the margin rule; it escalates 9.4 points more. The tax grows as payloads shrink: the more efficiently escalated images are encoded, the larger an always-on message is relative to them. The embedding it consumes carries no routing information beyond what the edge posterior already exposes for free.

Table 9: Ranking quality of escalation signals for identifying useful escalations (edge wrong, cloud right), MobileNetV3-L edge.

Signal	**AUC**
1− margin	**0.810**
1− maximum probability	0.807
Entropy	0.790
Cloud-supervised MLP verifier (best of 3 targets)	0.731

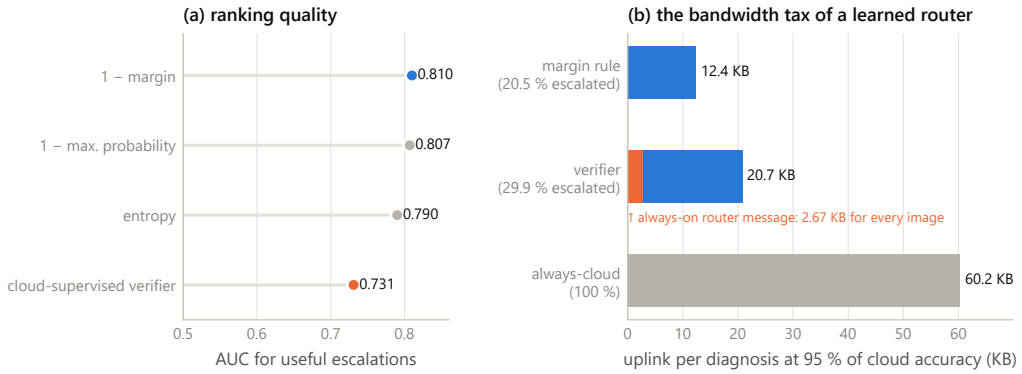


Figure 10: The cloud-supervised verifier against scalar signals. (a) AUC for identifying useful escalations. (b) Uplink per diagnosis at 95 % of cloud accuracy, read off the test curves: the verifier pays a 2.67 KB message for every image and still needs more escalations than the margin.

6.4. The ensemble as a teacher for the edge model

Section 6.1 showed that edge capacity sets the operating point. A more capable edge model need not be a larger one: the cloud ensemble can teach it. We tested this in two ways (Table 10).

Distillation before deployment. We retrained MobileNetV3-Large with the recipe and split of Section 3.3, adding to the loss the Kullback–Leibler divergence to the ensemble's class probabilities on each training image (Hinton et al., 2015) (temperature 2, weight 0.5). The probabilities come from the backbone features already computed for the training split, so the teacher adds no inference cost, and the distilled model keeps the architecture and on-device latency of the original. On its own it reaches 71.9 % top-1, against 69.4 % before distillation. With its threshold recalibrated on validation by the same rule (Section 3.4; $\tau = 0.073$), it escalates 12.5 % of test images instead of 18.7 % while delivering 94.0 % of cloud accuracy, so uplink traffic falls 8.0-fold instead of 5.3-fold; on the test curve the 95 % target is reached at 16.1 % escalation instead of 20.2 %.

Closing the loop after deployment. Every escalated image returns with the ensemble's verdict, a training signal the farm receives for free. To test whether it can keep improving the deployed edge model, we split the test set per class into a first "week" of 1 812 images, whose 344 escalated images are used to fine-tune the deployed model (eight short epochs, mixed with replayed training images), and a second week for evaluation. One week of escalations did not improve the edge model, whatever labels the escalated images carried, and the escalation needed for the target rose. A control that fine-tunes on replayed training images alone shows that part of this loss comes from the extra fine-tuning itself; but even farmer-confirmed labels do not help. The 344 images — about four per class, and the hardest the edge sees — are too few to move a model trained on 11 778 images of the same distribution. The value of the loop therefore lies in accumulating escalations over many weeks and in adapting to farms whose images differ from the training data, which an in-distribution benchmark cannot show; distilling the ensemble before deployment is, by contrast, an immediate gain.

6.5. On-device latency and energy

Table 11 reports latencies measured on the devices. On the Raspberry Pi 5, MobileNetV3-Large runs in 65.5 ms per image (fp32, 224×224, mean of 200 runs after warm-up, no thermal throttling). A FLOPs-based projection from the device's nominal throughput had predicted 32.1 ms; the measured figure is about twice as slow, which illustrates why projected latencies should not stand in for measurements on embedded targets. On the Jetson Nano the three backbones take 2 592, 2 673 and 1 613 ms, so an escalated image costs

Table 10: The ensemble as a teacher. Upper block: full test split, threshold calibrated on validation for each model (same evaluation pipeline for both; the original model's figures agree with Table 7 to within 0.1 point). Lower block: one-week loop pilot, evaluated on the second week; "escalation @95 %" is read off the test curve.

Edge model	**Edge top-1**	**Escal.**	**Top-1 (% cloud)**	**Uplink reduction**	**Escal. @95 %**
Original MobileNetV3-L	69.4 %	18.7 %	75.9 % (94.6)	5.3	20.2 %
Distilled from the ensemble	**71.9 %**	**12.5 %**	**75.4 % (94.0)**	**8.0**	**16.1 %**
Loop pilot: fine-tuning on one week's 344 escalated images, labelled by					
No update	69.1 %				18.9 %
No new images (control)	68.8 %				21.4 %
Ensemble, top-1 label	68.2 %				25.7 %
Ensemble, probabilities	68.2 %				23.3 %
Ensemble, confident only (130)	68.8 %				21.5 %
True label (farmer)	68.0 %				26.8 %

6.9 s of cloud computation including preprocessing and the head; loading the three backbones adds 36 s per batch, shared by all images in it. Because escalation is deferred, none of this is on the farmer's critical path: the farm answers in a median of 77 ms (95 % of images within 164 ms; 139 ms at the median for escalated images, which include the upload to the cloud queue), and the ensemble's confirmation arrives with the next batch.

Energy was measured on both devices. On the Pi, the power-management IC reports the voltage and current of every supply rail; over 300 edge inferences the SoC and memory rails drew 3.6 W on average (1.7 W at idle), or 0.45 J per image (0.24 J above idle). These rails exclude peripherals powered directly from the 5 V input, so the figure is a lower bound for the board. On the Nano, an escalated image costs 49 J at the board's input (Section 5.4). A diagnosis under selective escalation therefore costs 9.5 J of device energy (every image on the edge, 18.6 % of them also on the cloud node), against 49 J when every image is sent to the same cloud node: 5.1 times less, in line with the bandwidth saving.

6.6. Seven-day, four-farm replay

Over the replayed week the four farms submitted 3 677 images, and the edge escalated 684 of them (18.6 %) — the rate predicted offline for the same

Table 11: Latency measured on the embedded devices (single image, batch size 1). Replay values are medians over the seven-day replay.

Device	Stage	Latency
Raspberry Pi 5	MobileNetV3-L forward (fp32, 224 px), 200 runs	65.5 ms
	FLOPs-based projection, for reference	*32.1 ms*
	edge inference incl. preprocessing (replay)	70 ms
	answer on the farm, end to end (replay)	77 ms
	energy per image (SoC and memory rails)	0.45 J
Jetson Nano	DINOv2-L/14, 336 px (4 chunk engines, fp16)	2 592 ms
	DINOv3-L/16, 336 px (4 chunk engines, fp32)	2 673 ms
	CLIP-L/14, 224 px (4 chunk engines, fp32)	1 613 ms
	cloud compute per escalated image (replay)	6.9 s
	loading the three backbones, per batch (replay)	36 s
	energy per escalated image (board input)	49 J
Network	escalation upload to the cloud queue (replay)	69 ms

images at the same threshold (18.6 %), as expected from the edge node's agreement with the offline model (Table 5). The farms uploaded 40.4 MB in total, against 216.3 MB had every image been sent with the same payload: a 5.4-fold reduction in the bytes actually uploaded (Fig. 11). The offline model of Eq. (3) predicted 11.2 KB per diagnosis; the devices measured 11.2 KB. The system delivered 76.0 % correct diagnoses, against 69.3 % for the edge alone. On the escalated images, the Nano's ensemble was correct on 67.4 % of them; the workstation's offline predictions for the same images reach 66.5 %, and the two agree on 652/684 images (95.3 %): the on-device deviation changes individual diagnoses but not accuracy. The cloud node processed 14 batches of 48 images at the median, never received more than 14 escalations in a simulated hour, and confirmed every escalated image on the day it was captured (at most 6 simulated hours later). The whole replay took 98 minutes of wall-clock time, almost all of it spent on cloud computation.

The farms behave differently (Table 12, Fig. 13): farm B (solanaceae) escalated most often (24 %) and farm D (market garden) least (15 %), in line with the per-farm rates predicted offline from the margins of the same images. A single global threshold therefore translates into farm-specific uplink

budgets that follow how hard each farm's crops are for the edge model, which a deployment can monitor and, if needed, tune per farm. Figure 14 shows the operations console at the end of the replay.

Table 12: Measured results of the seven-day replay, per farm. Upload volumes are JPEG bytes actually sent; "Always-cloud" is the volume had every image been uploaded with the same payload (448 px shorter side, JPEG q90).

Farm	**Images**	**Escal.**	**Upload (MB)**	**Always-cloud (MB)**	**Reduction**	**Top-1**
A — orchard	1 360	18.9 %	14.6	75.3	5.1×	79.9 %
B — solanaceae	701	23.8 %	9.4	38.9	4.1×	62.2 %
C — field crops	707	17.8 %	7.7	43.6	5.6×	75.1 %
D — market garden	909	14.7 %	8.6	58.6	6.8×	81.6 %
All farms	3 677	18.6 %	40.4	216.3	5.4×	76.0 %

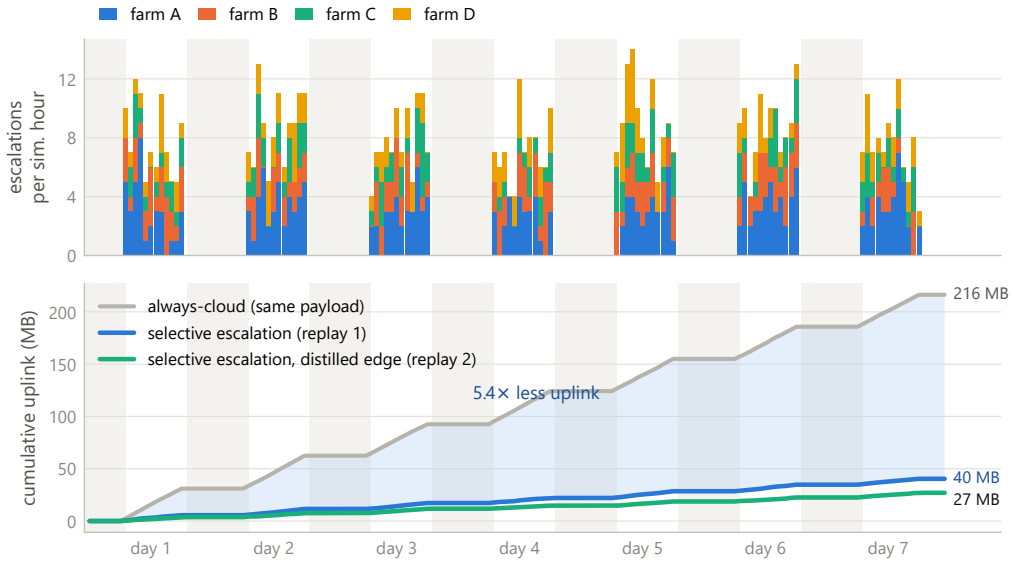


Figure 11: The seven-day replay on the embedded testbed. Top: escalations per simulated hour, by farm (shaded: night, no captures). Bottom: cumulative uploaded bytes, measured image by image, against the same images uploaded with the same payload by an always-cloud design; the green curve is the second replay, with the distilled edge model (Section 6.7).

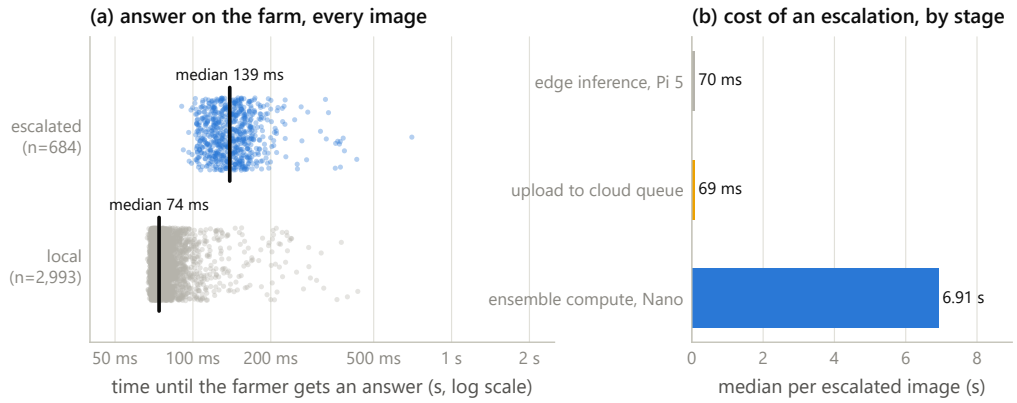


Figure 12: Measured latency during the replay. (a) Time until the farmer receives an answer, per image (log scale); escalated images are answered by the edge immediately (including the upload to the cloud queue) and confirmed by the cloud at the next batch. (b) Median cost of one escalation by stage: edge inference on the Pi, upload, and Nano compute (excluding the wait for the next batch and the per-batch model loading).

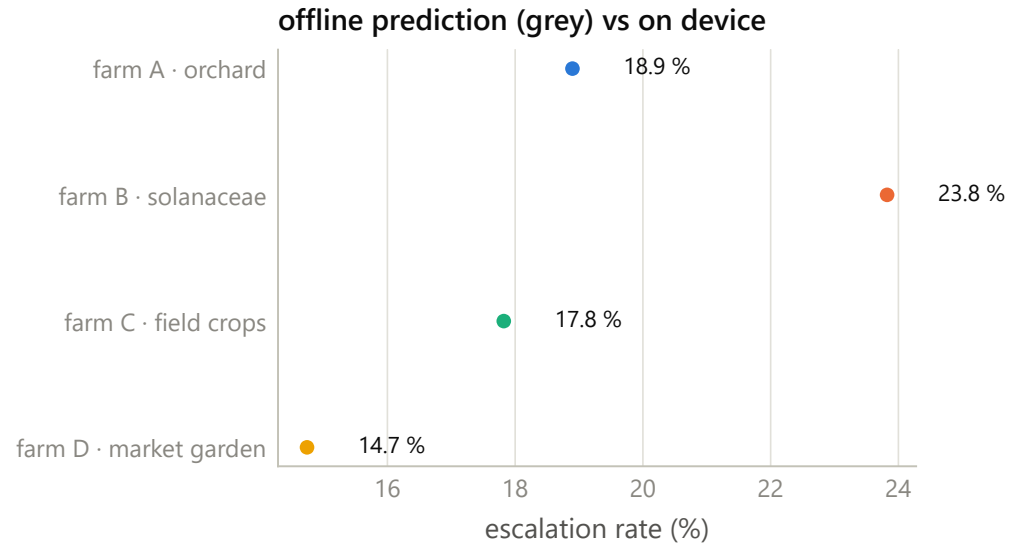


Figure 13: Escalation rate per farm: predicted from the offline margins (grey) and measured on the devices (colour).

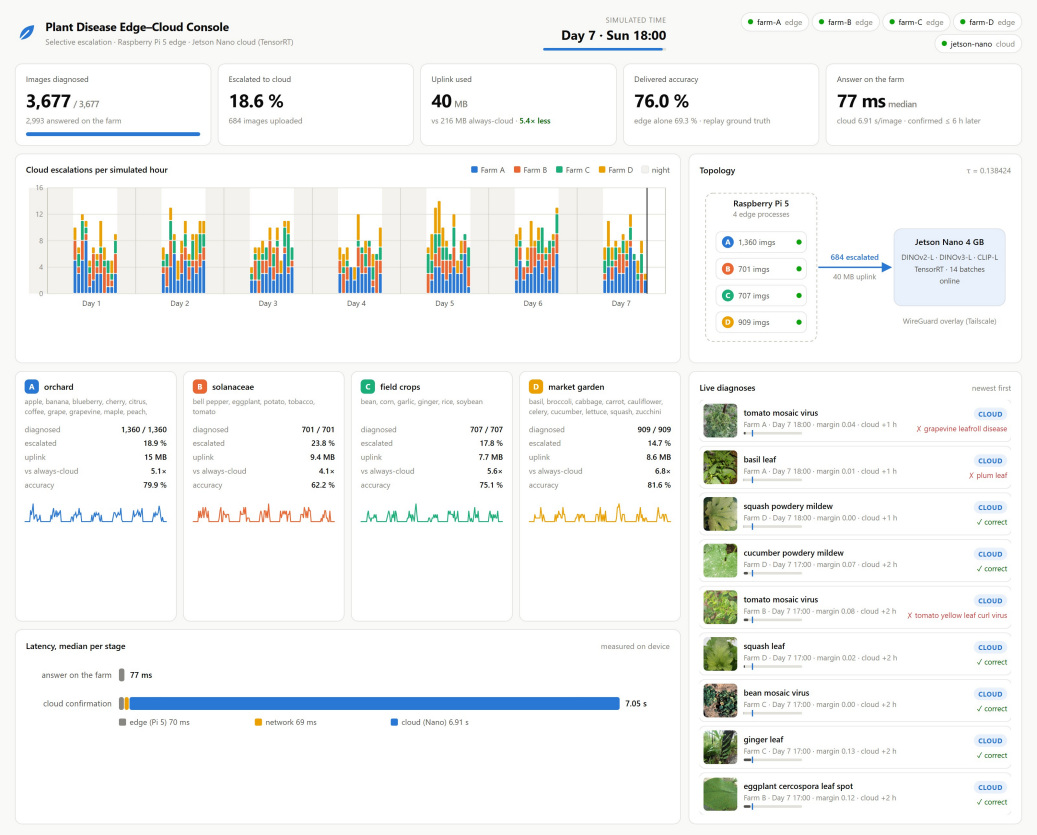


Figure 14: Operations console during the replay: simulated clock and node status, fleet indicators, escalations per simulated hour, per-farm cards, topology, latency by stage and a feed of the most recent diagnoses.

6.7. Improved configuration on the devices

The two improvements found after the first replay — the edge model distilled from the ensemble (Section 6.4) and CLIP in fp16 (Section 5) — change no interface, so we deployed them on the same devices (Appendix A, item C4) and repeated the seven-day replay with identical images, schedule, payload and batch times. The distilled model's threshold was recalibrated on validation by the rule of Section 3.4 ($\tau = 0.072$), and before the run the deployed edge reproduced the distilled model's offline predictions on 300 of 300 test images. Table 13 compares the two replays.

The edge escalated 455 images (12.4 %, against the offline prediction of 12.3 %), and the farms uploaded 27.0 MB instead of 40.4 MB: 7.5 KB per

diagnosis, an 8.0-fold reduction against always-cloud measured on the devices (Fig. 11). Every farm saved more than in the first replay, from 5.3-fold (solanaceae) to 11.3-fold (market garden). Delivered accuracy was 75.3 %, against 76.0 % in the first replay: both thresholds target 95 % of cloud accuracy on validation, so the stronger edge model spends its gain on bandwidth rather than on accuracy, trading a third of the uploaded bytes for under one point of accuracy. On the cloud node, fp16 CLIP shortened the compute per escalated image from 6.9 to 6.5 s, and the whole replay took 67 instead of 98 minutes of wall-clock time. With fewer and cheaper escalations, a diagnosis cost 6.4 J of device energy instead of 9.5 J, against 48 J for an always-cloud design on the same cloud node: 7.5 times less. The Nano's diagnoses agreed with the workstation's offline predictions on 429/455 escalated images.

Table 13: The two seven-day replays on the embedded testbed: identical images, schedule and payload; replay 2 uses the distilled edge model and fp16 CLIP. Device energy per diagnosis combines the Pi's energy per image with the Nano's energy per escalated image, measured in replay-sized batches of 48 (Section 5.4).

	Replay 1	**Replay 2**
Edge model	original	distilled
CLIP precision on the Nano	fp32	fp16
Threshold τ (calibrated on validation)	0.138	0.072
Images escalated	18.6 %	12.4 %
Uploaded (always-cloud: 216.3 MB)	40.4 MB	27.0 MB
Uplink reduction	5.4×	8.0×
Uplink per diagnosis	11.2 KB	7.5 KB
Delivered top-1	76.0 %	75.3 %
Edge model alone, top-1	69.3 %	71.9 %
Answer on the farm, median	77 ms	76 ms
Nano compute per escalated image, median	6.9 s	6.5 s
Model loading per batch, median	36 s	33 s
Batches (median size)	14 (48)	14 (34)
Device energy per diagnosis	9.5 J	6.4 J
Wall-clock time of the replay	98 min	67 min
Nano–workstation agreement, escalated images	652/684	429/455

7. Discussion

Strengthen the edge before engineering the router. The most consequential design decision in our study was not the escalation rule but the edge model. Moving from MobileNetV3-Small to MobileNetV3-Large cost 2.7× the parameters and 3.8× the operations — still only 65.5 ms on a Raspberry Pi 5 — and halved the escalation rate, whereas a learned edge-side router saved only two points of escalation and a cloud-supervised one did worse than the training-free scalar. Capacity can also be added without changing the architecture: distilling the ensemble into the same MobileNetV3-Large cut the calibrated escalation rate from 18.7 to 12.5 % (Section 6.4), and on the devices from 18.6 % to 12.4 %, an 8.0-fold uplink saving (Section 6.7). The reason is informational: an edge-side router can only re-express the edge model's posterior, and the margin already summarises it well. The oracle shows that substantial headroom remains (84.8 % at 20 % escalation), but reaching it requires information the edge does not have; a cheaper path to it is a stronger edge model.

What the bandwidth axis reveals. Latency-centred evaluations treat an always-on router message as negligible; on the bandwidth axis it is a fixed tax that a learned router must earn back through fewer escalations. For field deployments on metered cellular links the relevant figure is the uplink volume per farm and month. At the measured 11.2 KB per diagnosis, a farm capturing 100 images per day would upload 33 MB per month, against 177 MB for always-cloud with the same payload; on a 1 Mbit/s uplink an escalated image takes about 0.5 s to transmit.

A cloud that fits on a farm. Serving a 0.9-billion-parameter ensemble from a 4 GB Jetson Nano shows that the "cloud" in selective escalation need not be a data centre. A cooperative, an extension office or a large farm can host the large model locally, which keeps field images within the community's network, removes the dependency on wide-area connectivity for the cloud tier, and turns a recurring service cost into a one-off hardware cost. It does not come for free. Each escalated image costs 6.9 s of Nano compute, and because the full ensemble does not fit in memory, the cloud tier must work in batches; in the replay a batch of 48 images also paid 36 s to load the three backbones, and the board spends 49 J per escalated image (Section 5.4). Selective escalation is what makes this affordable: at 18–19 % escalation the Nano sees one image in five, whereas an always-cloud design would multiply

its workload, and its energy use, more than fivefold. Beyond this study, the port shows that hardware the software ecosystem has left behind can still serve current foundation models, provided memory, precision and fidelity are engineered explicitly; the same five principles apply to other legacy accelerators that farms and cooperatives already own. The batched design suits scouting and monitoring workflows, in which a confirmed diagnosis within hours is valuable but an instant one is not required; the farmer always receives the edge's answer immediately. A node with more memory, such as an 8 GB module, could keep all engines resident and answer synchronously; the server implements this mode, but we have not tested it.

Portability is more than the model. Two of the deviations we measured did not come from the models at all. The edge server initially resized images bicubically while the edge model had been trained with bilinear resizing, and the Nano's older image library decodes JPEGs differently (Section 5.5). Both shifted predictions without any error being raised. Verifying deployed nodes against offline predictions image by image, as in Table 5, is what exposed them; we recommend the practice for any edge–cloud deployment.

Limitations. Our study uses a single, if diverse, benchmark; PlantWild's class balance and capture conditions may differ from a given region. The escalation threshold is global; per-crop thresholds could equalise accuracy across farms. The replay compresses idle time and therefore does not capture diurnal effects, power cycling or connectivity outages, and its four farms share one physical Pi, so device-to-device variability is not represented. Each edge model was trained once, so we report no variance across training seeds; the same holds for the distillation and loop experiments. The Pi's energy covers its SoC and memory rails only and is therefore a lower bound. In our testbed the devices share a local network, so measured network latencies are a best case for rural links; transfer times on slower links follow directly from the measured byte counts. The confirmation delay of the batched cloud tier is a design parameter set in simulated time (two batches a day), not a measured quantity; its compute and loading costs are measured. Finally, the Nano's cloud pipeline deviates slightly from the workstation's, chiefly through its older image library; Table 5 and the replay-wide agreement quantify the deviation.

8. Conclusion

Selective escalation reconciles the accuracy of foundation-model ensembles with the connectivity of real farms. A MobileNetV3-Large edge model that forwards an image only when its two leading hypotheses are close — with a threshold fixed on validation data — retains 94.6 % of a three-backbone ensemble's accuracy on held-out in-the-wild plant disease images while uploading 5.4 times less data. Edge-model capacity, not router complexity, sets this trade-off, and a cloud-supervised verifier pays for its own message without improving routing. The complete system runs on a Raspberry Pi 5 and an end-of-life 4 GB Jetson Nano, made possible by a port that decouples the ensemble from its framework and runs it one chunk engine at a time, at 49 J per escalated image; a seven-day, four-farm replay on that hardware confirms the offline trade-off: 18.6 % of images escalated, uploaded bytes cut 5.4-fold, and 76.0 % of diagnoses correct, with a median answer time on the farm of 77 ms and 5.1 times less device energy per diagnosis than always-cloud. Distilling the ensemble into the edge model, at no inference cost, raises the uplink saving measured on the same devices from 5.4- to 8.0-fold (7.5 KB per diagnosis) at 75.3 % delivered accuracy. Future work will accumulate escalations over full seasons to test whether the loop from cloud to edge improves farms whose images differ from the training data, a gain one week of in-distribution escalations could not show; it will also tune thresholds per crop and study connectivity outages over full-length deployments.

CRediT authorship contribution statement

Youssef Natij, Hajar El Karch, Abdelkader Mezouari: Conceptualization, Methodology, Software, Validation, Investigation, Formal analysis, Writing – original draft, Writing – review & editing. All authors contributed equally to the conception, implementation, analysis and writing of this work.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Data availability

PlantWild is available from its authors (Wei et al., 2024b) under the terms of their release; this work does not redistribute its images. Everything needed to reproduce the tables and figures of this paper is in the project repository, `https://github.com/YouROS12/CloudOffEdge`: the deployment code for the Raspberry Pi and the Jetson Nano, the porting toolkit (chunked ONNX export, fp16 conversion, chunk-by-chunk TensorRT build, chunk-major runtime), the fleet emulator and operations console, the training, distillation, calibration and measurement scripts, the trained edge models and cloud head, the per-image predictions, the per-image logs of both replays, the activation, memory and power profiles, the TensorRT verification reports and the sources of this manuscript. The repository is private during review; reviewers can obtain access from the corresponding author, and it will be made public upon publication.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the authors used Claude (Anthropic) to assist with software development and with drafting and editing the manuscript. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

Appendix A. Software adaptations for embedded deployment

The reference pipeline was developed on a workstation (Python 3.12, PyTorch 2.11, `timm` 1.0.26, `open_clip` 3.3.0). Running it on the embedded testbed required the adaptations below, listed so that the deployment can be reproduced on the same hardware generation. Each adaptation was verified against the reference pipeline (Table 5).

Table A.14: Software adaptations, by node.

#	Problem and cause	Adaptation
Edge node — Raspberry Pi 5, Debian 13, Python 3.13		
A1	Reference dependency pins (PyTorch 2.2, NumPy 1.26) have no Python 3.13 wheels for ARM64.	Version-conditional pins (PEP 508 markers): original pins below 3.13, PyTorch 2.6 / NumPy 2.2 from 3.13.
A2	One physical device must emulate several farms.	systemd template unit; one independent server process per farm with its own identifier and port.
A3	Latency reported only as a total.	Edge reports its own inference time; the cloud reports per-backbone times; network time is the remainder.
A4	A cloud outage would appear as a low escalation rate.	Escalation decisions logged separately from cloud success; failed escalations fall back to the edge label.
A5	The edge server resized images bicubically, whereas the model was trained and evaluated with bilinear resizing, which shifts the margins.	Bilinear resizing restored; the deployed node reproduces the offline prediction on 300/300 test images.
A6	Escalated images were uploaded at camera resolution (up to $\sim$6 MB), although the cloud works at $\leq$336 px.	Payload resized to a 448 px shorter side, JPEG q90 (Table 2); the always-cloud baseline uses the same payload.
A7	A failed escalation was billed as uploaded bytes.	Bytes are counted only once the cloud acknowledges the image.
Cloud node — Jetson Nano 4 GB, JetPack 4.6.1, Python 3.6, CUDA 10.2, TensorRT 8.2.1		
B1	The reference server needs Python $\geq$3.8 (PyTorch 2, `timm` for DINOv3), unavailable on JetPack 4.	Model decoupled from framework: ONNX export on the workstation, TensorRT engines built on the Nano, torch-free Python 3.6 server.
B2	TensorRT 8.2 parses ONNX opsets up to 13.	Export at opset 13 with the TorchScript-based exporter.
B3	Fused scaled-dot-product attention has no opset-13 mapping.	Explicit softmax$(QK^\top/\sqrt{d})V$ substituted during export (mathematically identical).

#	Problem and cause	Adaptation
B4	The `nn.MultiheadAttention` fast path (CLIP) emits a fused operator with no ONNX mapping.	Fast path disabled during export.
B5	DINOv2 bicubically resizes its position grid on every forward; TensorRT 8.2 has no cubic resize.	Input size is fixed, so the interpolated position embedding is computed once in PyTorch and stored as a constant.
B6	DINOv3 recomputes its rotary position embedding from the input shape on every forward, and CLIP's reshapes emit dynamic shape arithmetic; both export as conditionals (`If`) and modulo operators.	Rotary embedding baked as a constant for the fixed 21×21 patch grid; remaining shape arithmetic constant-folded (onnx-simplifier); export rejected if an unsupported operator remains.
B7	No PyTorch/torchvision in the serving process.	NumPy/Pillow re-implementation of preprocessing (bit-identical) and of the QualityGate head (logits within 3×10^{-6}).
B8	Engine builds were killed for lack of memory: 4 GB is shared by CPU and GPU, and the stock swap is compressed RAM, in which weights barely compress.	One engine built per process; the graph parsed directly from file (one copy of the weights); fp16 with a 256 MB builder workspace; a temporary swap file on the SD card for the build only.
B9	Even so, building from fp32 graphs exhausted the memory the GPU can allocate (which cannot be swapped).	Graphs converted to fp16 weights on the workstation (halving them to ∼607 MB), with LayerNorm arithmetic kept in fp32 because ViT activation outliers overflow fp16 when squared; cosine to the fp32 reference ≥ 0.9999.
B10	A whole fp16 ViT-L still could not be built: parsing alone peaks at 2.75 GB, and the build thrashed swap with the GPU idle.	Each backbone exported as four chunks of six transformer blocks, built one engine at a time and chained at run time (exact: cosine 1.000000 to the unsplit model).

#	Problem and cause	Adaptation
B11	TensorRT 8.2 compiled each chunk into one fused kernel graph holding 969 MB of device memory (13 bytes per parameter).	Every linear layer exported as an equivalent 1×1 convolution (attention for CLIP rewritten explicitly); engine size 612 → 157 MB, no fused graphs remain.
B12	Chunk builds took ∼10 min each.	A timing cache shared across chunks, whose layer shapes are identical within a backbone.
B13	Only nine of the twelve chunk engines fit in memory, even with one shared activation buffer.	Chunk-major batched cloud inference with deferred escalation (Section 5.4); one chunk engine resident at a time.
B14	First parity check failed (top-1 agreement 1/24): DINOv3 features were NaN. Pinning LayerNorm, then all non-convolution layers, to fp32 changed nothing; profiling showed DINOv3's residual stream reaching 1.6×10^5, beyond the fp16 range.	Per-backbone precision: DINOv2 fp16, DINOv3 fp32; CLIP also fp32, chosen while its deviation (B15) was still attributed to precision (a later check showed fp16 CLIP equivalent and faster). Every layer pinned with TensorRT precision constraints, since the fp16-weight graph requires the fp16 builder flag.
B15	CLIP still deviated (cosine below 0.99 on 5/24 reference images), identically in fp16 and fp32.	Traced to the image library: Pillow 8.4 (the last release for Python 3.6) decodes and resamples JPEGs differently from Pillow 12. Given identical input tensors the engines match the reference (cosine 0.99999–1.00000, 24/24 top-1). Not correctable on JetPack 4; documented and measured over the replay.
B16	TensorRT Python bindings ship with JetPack, not PyPI.	Virtual environment with access to system site-packages.
B17	Current Python packages no longer support Python 3.6.	Last compatible releases pinned (NumPy 1.19.5, Pillow 8.4.0, pycuda 2021.1, FastAPI 0.83, pydantic 1.9).

#	Problem and cause	Adaptation
B18	`import numpy` aborts with an illegal instruction: the bundled OpenBLAS mis-detects the Cortex-A57.	`OPENBLAS_CORETYPE=ARMV8` set for every process.
B19	pycuda's isolated build compiles NumPy from source (∼20 min on the Nano).	NumPy installed from a wheel first, pycuda built without build isolation.
B20	The shared message definitions require Python ≥3.7.	Message models mirrored in pydantic v1; the wire format is unchanged, so edge nodes need no change.
B21	The web server runs requests on a thread pool; CUDA contexts are thread-bound.	One CUDA context shared by all engines, pushed and popped around each inference under a lock.
Workstation and orchestration		
C1	Scripts edited on Windows carried CRLF line endings, which break Bash on the devices.	LF enforced through `.gitattributes` and normalised before shipping.
C2	Devices hold no credentials for the private code repository.	Code shipped over SSH from the workstation; checkpoint integrity checked by SHA-256.
C3	The paper's test split had to be replayed exactly.	The seed-fixed 3 677-image split materialised as hard links in a class-per-folder layout.
C4	A second, improved configuration (Section 6.7) had to run on the same devices without losing the first.	Distilled edge checkpoint and its own validation-calibrated threshold selected through a separate configuration file; fp16 CLIP engines in a separate engine directory selected by a service override; both reversible. The deployed edge reproduces the distilled model's offline predictions on 300/300 test images.

References

Ahmad, A., Gamal, A.E., Saraswat, D., 2023. Toward generalization of deep learning-based plant disease identification under controlled and field conditions. IEEE Access 11, 9042–9057. doi:10.1109/ACCESS.2023.3240100.

Bai, J., Lu, F., Zhang, K., et al., 2019. ONNX: Open neural network exchange. https://github.com/onnx/onnx. Accessed 2026-10-11.

Barbedo, J.G.A., 2018. Impact of dataset size and variety on the effectiveness of deep learning and transfer learning for plant disease classification. Computers and Electronics in Agriculture 153, 46–53. doi:10.1016/j.compag.2018.08.013.

Chow, C.K., 1970. On optimum recognition error and reject tradeoff. IEEE Transactions on Information Theory 16, 41–46. doi:10.1109/TIT.1970.1054406.

Donenfeld, J.A., 2017. WireGuard: Next generation kernel network tunnel, in: Network and Distributed System Security Symposium (NDSS). doi:10.14722/ndss.2017.23160.

El Karch, H., Natij, Y., Benaly, M., Elgouri, R., Mezouari, A., 2025. Backbone diversity beats text supervision: A systematic study of frozen multi-foundation-model fusion for in-the-wild plant disease recognition. Manuscript under review.

Ferentinos, K.P., 2018. Deep learning models for plant disease detection and diagnosis. Computers and Electronics in Agriculture 145, 311–318. doi:10.1016/j.compag.2018.01.009.

Geifman, Y., El-Yaniv, R., 2017. Selective classification for deep neural networks, in: Advances in Neural Information Processing Systems (NeurIPS).

Guo, C., Pleiss, G., Sun, Y., Weinberger, K.Q., 2017. On calibration of modern neural networks, in: Proceedings of the 34th International Conference on Machine Learning (ICML), pp. 1321–1330.

Hasan, M.J., Mazumdar, S., Momen, S., 2025. Deployable deep learning for cross-domain plant leaf disease detection via ensemble learning, knowledge distillation, and quantization. IEEE Access 13, 140313–140336. doi:10.1109/ACCESS.2025.3595390.

Hendrycks, D., Gimpel, K., 2017. A baseline for detecting misclassified and out-of-distribution examples in neural networks, in: International Conference on Learning Representations (ICLR).

Hinton, G., Vinyals, O., Dean, J., 2015. Distilling the knowledge in a neural network. doi:10.48550/arXiv.1503.02531. arXiv preprint arXiv:1503.02531.

Howard, A., Sandler, M., Chu, G., Chen, L.C., Chen, B., Tan, M., Wang, W., Zhu, Y., Pang, R., Vasudevan, V., Le, Q.V., Adam, H., 2019. Searching for MobileNetV3, in: IEEE/CVF International Conference on Computer Vision (ICCV), pp. 1314–1324. doi:10.1109/ICCV.2019.00140.

Hughes, D.P., Salathé, M., 2015. An open access repository of images on plant health to enable the development of mobile disease diagnostics. doi:10.48550/arXiv.1511.08060. arXiv preprint arXiv:1511.08060.

Jiang, X., Wang, J., Xie, K., Cui, C., Du, A., Shi, X., Yang, W., Zhai, R., 2025. PlantCaFo: An efficient few-shot plant disease recognition method based on foundation models. Plant Phenomics 7, 100024. doi:10.1016/j.plaphe.2025.100024.

Kag, A., Fedorov, I., Gangrade, A., Whatmough, P., Saligrama, V., 2023. Efficient edge inference by selective query, in: International Conference on Learning Representations (ICLR).

Kamilaris, A., Prenafeta-Boldú, F.X., 2018. Deep learning in agriculture: A survey. Computers and Electronics in Agriculture 147, 70–90. doi:10.1016/j.compag.2018.02.016.

Kang, Y., Hauswald, J., Gao, C., Rovinski, A., Mudge, T., Mars, J., Tang, L., 2017. Neurosurgeon: Collaborative intelligence between the cloud and mobile edge, in: Proceedings of the 22nd International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), pp. 615–629. doi:10.1145/3037697.3037698.

Lin, T.Y., Goyal, P., Girshick, R., He, K., Dollár, P., 2017. Focal loss for dense object detection, in: IEEE International Conference on Computer Vision (ICCV), pp. 2999–3007. doi:10.1109/ICCV.2017.324.

Loshchilov, I., Hutter, F., 2019. Decoupled weight decay regularization, in: International Conference on Learning Representations (ICLR).

Matsubara, Y., Levorato, M., Restuccia, F., 2022. Split computing and early exiting for deep learning applications: Survey and research challenges. ACM Computing Surveys 55, 1–30. doi:10.1145/3527155.

Mohanty, S.P., Hughes, D.P., Salathé, M., 2016. Using deep learning for image-based plant disease detection. Frontiers in Plant Science 7, 1419. doi:10.3389/fpls.2016.01419.

Noyan, M.A., 2022. Uncovering bias in the PlantVillage dataset. doi:10.48550/arXiv.2206.04374. arXiv preprint arXiv:2206.04374.

NVIDIA Corporation, 2022. NVIDIA TensorRT 8.2 developer guide. https://docs.nvidia.com/deeplearning/tensorrt/archives/. Accessed 2026-10-11.

O'Grady, M.J., Langton, D., O'Hare, G.M.P., 2019. Edge computing: A tractable model for smart agriculture? Artificial Intelligence in Agriculture 3, 42–51. doi:10.1016/j.aiia.2019.12.001.

Oquab, M., Darcet, T., Moutakanni, T., Vo, H.V., Szafraniec, M., Khalidov, V., Fernandez, P., Haziza, D., Massa, F., El-Nouby, A., et al., 2024. DINOv2: Learning robust visual features without supervision. Transactions on Machine Learning Research (TMLR).

Radford, A., Kim, J.W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., Sutskever, I., 2021. Learning transferable visual models from natural language supervision, in: International Conference on Machine Learning (ICML), pp. 8748–8763.

Salman, Z., Muhammad, A., Han, D., 2025. Plant disease classification in the wild using vision transformers and mixture of experts. Frontiers in Plant Science 16. doi:10.3389/fpls.2025.1522985.

Savary, S., Willocquet, L., Pethybridge, S.J., Esker, P., McRoberts, N., Nelson, A., 2019. The global burden of pathogens and pests on major food crops. Nature Ecology & Evolution 3, 430–439. doi:10.1038/s41559-018-0793-y.

Shi, W., Cao, J., Zhang, Q., Li, Y., Xu, L., 2016. Edge computing: Vision and challenges. IEEE Internet of Things Journal 3, 637–646. doi:10.1109/JIOT.2016.2579198.

Siméoni, O., Vo, H.V., Seitzer, M., Baldassarre, F., Oquab, M., et al., 2026. DINOv3: A versatile vision foundation model. Transactions on Machine Learning Research (TMLR). URL: https://openreview.net/forum?id=2NlGyqNjns.

Singh, D., Jain, N., Jain, P., Kayal, P., Kumawat, S., Batra, N., 2020. PlantDoc: A dataset for visual plant disease detection, in: Proceedings of the 7th ACM IKDD CoDS and 25th COMAD, pp. 249–253. doi:10.1145/3371158.3371196.

Tan, M., Le, Q.V., 2019. EfficientNet: Rethinking model scaling for convolutional neural networks, in: International Conference on Machine Learning (ICML), pp. 6105–6114.

Teerapittayanon, S., McDanel, B., Kung, H.T., 2016. BranchyNet: Fast inference via early exiting from deep neural networks, in: 23rd International Conference on Pattern Recognition (ICPR), pp. 2464–2469. doi:10.1109/ICPR.2016.7900006.

Teerapittayanon, S., McDanel, B., Kung, H.T., 2017. Distributed deep neural networks over the cloud, the edge and end devices, in: IEEE 37th International Conference on Distributed Computing Systems (ICDCS), pp. 328–339. doi:10.1109/ICDCS.2017.226.

Viola, P., Jones, M., 2001. Rapid object detection using a boosted cascade of simple features, in: Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR), pp. I–511–I–518. doi:10.1109/CVPR.2001.990517.

Wang, X., Luo, Y., Crankshaw, D., Tumanov, A., Yu, F., Gonzalez, J.E., 2018. IDK cascades: Fast deep learning by learning not to overthink, in: Proceedings of the Conference on Uncertainty in Artificial Intelligence (UAI).

Wei, T., Chen, Z., Huang, Z., Yu, X., 2024a. Benchmarking in-the-wild multimodal disease recognition and a versatile baseline, in: Proceedings

of the 32nd ACM International Conference on Multimedia (MM '24), pp. 1593–1601. doi:10.1145/3664647.3680599.

Wei, T., Chen, Z., Huang, Z., Yu, X., 2024b. Plant disease recognition in the wild, in: Proceedings of the 32nd ACM International Conference on Multimedia (MM '24). doi:10.1145/3664647.3681614.

Wolfert, S., Ge, L., Verdouw, C., Bogaardt, M.J., 2017. Big data in smart farming – a review. Agricultural Systems 153, 69–80. doi:10.1016/j.agsy.2017.01.023.

Wu, X., Fan, X., Luo, P., Choudhury, S.D., Tjahjadi, T., Hu, C., 2023. From laboratory to field: Unsupervised domain adaptation for plant disease recognition in the wild. Plant Phenomics 5, 0038. doi:10.34133/plantphenomics.0038.